- Mo-Lights Chess - Complete Documentation
- 1. Introduction
- 2. Quick Start
  - Quick Commands
  - Quick Command Reference
- 3. Installation Guide
  - 3.1 Windows Installation
  - 3.2 macOS Installation
  - 3.3 Linux Installation
  - Debian/Ubuntu
  - Arch Linux
  - Fedora/RHEL
  - Source Compilation
  - Verify Installation
  - Troubleshooting
  - Uninstallation
  - Portable Mode
  - Dependencies
  - Next Steps
- 4. User Guide
  - 4.1 Playing Your First Game
  - Starting a New Game
  - Making Your First Move
  - Understanding the Board
  - Board Coordinates
  - 4.2 Understanding the Board
  - Board Coordinates
  - 4.3 Making Moves
  - Algebraic Notation (Recommended)
  - Form Moves
  - Coordinate Notation (Alternative)
  - 4.4 Special Moves
  - Castling
  - Kingside Castling
  - Queenside Castling
  - Promotion
  - Promotion to Queen: Options: Q (Queen (most common)), R (Rook), B (Bishop), N (Knight)
  - 4.5 Game Commands
  - Command Reference
  - Resign the game
  - Offer a draw
  - Accept draw offer
  - Decline draw offer
  - Request undo

Saving and Loading

```
// Save
PGNWriter writer = new PGNWriter(engine);
writer.writePGN("game.pgn", "Casual Game", "Alice", "Bob", "*");

// Load
ChessEngine newEngine = new ChessEngine();
PGNReader reader = new PGNReader(newEngine);
reader.readPGN("game.pgn");
```

---

6. Developer Guide

6.1 Project Structure

```
Mo-Lights-Chess/
├── Main.java                   # CLI interface
├── ChessEngine.java            # Game logic
├── Board.java                  # Board state
├── Square.java                 # Chess square
├── Move.java                   # Move representation
├── GameResult.java             # Game outcome
├── AlgebraicNotationParser.java # SAN parser
├── PGNReader.java              # PGN reader
├── PGNWriter.java              # PGN writer
├── pieces/
│   ├── Piece.java              # Abstract base
│   ├── Pawn.java
│   ├── Rook.java
│   ├── Knight.java
│   ├── Bishop.java
│   ├── Queen.java
│   └── King.java
└── exceptions/
    ├── InvalidFileException.java
    ├── GameStateException.java
    ├── InvalidColorException.java
    ├── InvalidMoveException.java
    ├── InvalidSquareException.java
    ├── NotImplemented.java
    └── WrongPieceException.java
```

6.2 Architecture Overview

Data Flow

```
User Input → Main → AlgebraicNotationParser → ChessEngine → Board → Pieces
```

Move Validation Pipeline

1. **getAllPseudoMoves()** - Generate moves based on piece movement rules
2. **getAllLegalMoves()** - Filter moves by testing each:
   - Temporarily make the move
   - Check if own King is in check
   - Undo the move
   - Keep only safe moves

6.3 Design Patterns

Template Method Pattern (Pieces)

```
public abstract class Piece {
    public abstract List<int[]> getPossibleMoves(int fromRow, int fromCol);
}

public class Knight extends Piece {
    @Override
    public List<int[]> getPossibleMoves(int fromRow, int fromCol) {
        // Knight-specific movement logic
    }
}
```

Command Pattern (Moves)

Each `Move` object encapsulates: - What piece moved - Where it moved - What was captured - Special move flags

Enables undo/redo functionality.

Facade Pattern (ChessEngine)

`ChessEngine` provides simplified interface to complex chess rules:

```
engine.makeMove(move);
engine.inCheckMate();
engine.getAllLegalMoves();
```

6.4 Adding New Features

Adding a New Piece Type

1. Create the piece class in `pieces/`:

```
package pieces;

public class Archbishop extends Piece {
    public Archbishop(String color) {
        super(color);
    }

    @Override
    public List<int[]> getPossibleMoves(int fromRow, int fromCol) {
        // Implement movement (Bishop + Knight)
        return moves;
    }

    @Override
    public String getSymbol() {
        return color.equals("white") ? "♗" : "♝";
    }
}
```

2. Add to `ChessEngine.getAllPossibleMoves()`:

```
if(currentPiece.getType().equals("Archbishop"): {
    this.getArchbishopMoves(currentSquare, this.board, possibleMoves);
}
```

3. Implement move generation method:

```
public void getArchbishopMoves(Square startSquare, Board board,
                               List<Move> possibleMoves) {
    // Combine bishop and knight move logic
}
```

Adding a New Command

In `Main.java`, add to the switch statement:

```
case "hint":
    List<Move> legalMoves = engine.getAllLegalMoves();
    if (!legalMoves.isEmpty()) {
        Move randomMove = legalMoves.get(
            new Random().nextInt(legalMoves.size()
        );
        System.out.println("Try: " + randomMove);
    }
    break;
```

6.5 Code Style Guidelines

Naming Conventions

```
// Classes: PascalCase
public class ChessEngine { }

// Methods: camelCase
public void makeMove(Move move) { }

// Variables: camelCase
private String currentTurn;

// Constants: UPPER_SNAKE_CASE
private static final int BOARD_SIZE = 8;
```

Method Structure

```
/**
 * Javadoc comment for public methods
 *
 * @param row The row index (0-7)
 * @return true if successful
 */
public boolean hasRow(int row) {
    // Keep methods short: <= 50 lines)
    // Single responsibility
    return true;
}
```

6.6 Testing

Manual Test Checklist

- [ ] Basic moves for all pieces
- [ ] Castling (both sides, blocked scenarios)
- [ ] En passant
- [ ] Pawn promotion
- [ ] Check detection
- [ ] Checkmate detection
- [ ] Stalemate detection
- [ ] Undo functionality
- [ ] Save/load PGN files

Test PGN File

```
[Event "Test"]
[Site "Local"]
[Date "2025.01.01"]
[Round "1"]
[White "Player1"]
[Black "Player2"]
[Result "*"]

1. e4 e5 2. Nf3 Nc6 3. Bb5 *
```

6.7 Performance Considerations

Operation	Complexity
getPossibleMoves()	O(n) where n = pieces
getAllLegalMoves()	O(m × k) where m = possible moves, k = opponent pieces
makeMove()	O(1)
undoMove()	O(1)

6.8 Debugging Tips

Common Issues

"King not found" exceptions: - Check makeMove/undoMove logic - Verify board state integrity

Empty legal moves when shouldn't be: - Debug isInCheck() and isSquareUnderAttack() - Add logging in getAllLegalMoves()

Castling not allowed: - Check king/rook has Moved flags - Verify path is clear - Ensure not castling through check

Debug Output Example

```
// Add to getAllLegalMoves() for debugging
System.out.println("Possible moves: " + possibleMoves.size());
for (Move m : possibleMoves) {
    System.out.println("  " + m);
}
```

---

7. Quick Reference

7.1 Commands

Command	Action
`e4`	Pawn to e4
`Nf3`	Knight to f3
`Bxe5`	Bishop captures e5
`O-O`	Castle kingside
`O-O-O`	Castle queenside
`e8=Q`	Promote to queen
`e2 e4`	Coordinate notation
`moves`	Show legal moves
`undo`	Take back move
`save game`	Save to game.pgn
`resign`	Give up
`draw`	Offer draw
`accept`	Accept draw
`help`	Show help
`menu`	Main menu

7.2 Piece Symbols

Piece	White	Black	Letter	Value
King	♔	♚	K	∞
Queen	♕	♛	Q	9
Rook	♖	♜	R	5
Bishop	♗	♝	B	3
Knight	♘	♞	N	3
Pawn	♙	♟	-	1

7.3 Board Diagram

```
  a b c d e f g h
8 ♜ ♞ ♝ ♛ ♚ ♝ ♞ ♜
7 ♟ ♟ ♟ ♟ ♟ ♟ ♟ ♟
6 · · · · · · · ·
5 · · · · · · · ·
4 · · · · · · · ·
3 · · · · · · · ·
2 ♙ ♙ ♙ ♙ ♙ ♙ ♙ ♙
1 ♖ ♘ ♗ ♕ ♔ ♗ ♘ ♖
```

7.4 Notation Examples

Algebraic	Coordinate	Meaning
`e4`	`e2 e4`	Pawn to e4
`Nf3`	`g1 f3`	Knight to f3
`Bxe6`	`b5 e6`	Bishop captures e6
`O-O`	-	Kingside castle
`O-O-O`	-	Queenside castle
`e8=Q`	`e7 e8`	Promote to queen
`exd5`	`e4 d5`	Pawn captures d5
`Rbd7`	`b8 d7`	b-Knight to d7

7.5 Special Moves

Castling Requirements

- ✓ King hasn't moved
- ✓ Rook hasn't moved
- ✓ No pieces between
- ✓ King not in check
- ✓ King doesn't pass through check
- ✓ King doesn't land in check

En Passant Requirements

- ✓ Your pawn on 5th rank (White) or 4th rank (Black)
- ✓ Enemy pawn just moved 2 squares
- ✓ Enemy pawn beside your pawn
- ✓ Must capture immediately

Pawn Promotion Options

- `e8=Q` → Queen (recommended)
- `e8=R` → Rook
- `e8=B` → Bishop
- `e8=N` → Knight

7.6 Game States

State	Description
Check	King is attacked
Checkmate	King in check, no escape (lose)
Stalemate	No legal moves, not in check (draw)
Resignation	Player gives up
Draw	Both players agree

7.7 Opening Principles

1. Control the center (e4, d4, e5, d5)
2. Develop pieces (knights & bishops)
3. Castle early (protect king)
4. Don't move same piece twice
5. Connect rooks

7.8 Troubleshooting Quick Fixes

Problem	Solution
Can't compile	Check you're in project root
Can't run	Verify Main.class exists
Java not found	Add Java to PATH
Pieces show as boxes	Enable UTF-8: `chcp 65001` (Windows)
Illegal move error	Use `moves` to see legal options
Can't castle	Check requirements above

7.9 File Locations

```
Mo-Lights-Chess/
├── *.java        # Source files
├── *.class       # Compiled files
└── *.pgn         # Saved games
    docs/         # Documentation
```

Appendix

A. Project Information

Project: Mo-Lights Chess
Language: Java
Minimum Java Version: JDK 11
License: Open Source
Repository: https://github.com/yoursername/Mo-Lights-Chess

B. Credits

Chess Rules: FIDE (World Chess Federation)
PGN Format: Standard Portable Game Notation
Unicode Symbols: Chess piece Unicode characters

C. Version History

Version 1.0 (January 2025) - Initial release - Complete chess rules implementation - PGN support - Algebraic notation - Save/load functionality - Draw and resignation features

---

End of Documentation

For the latest updates and support, visit the GitHub repository.

Enjoy playing Mo-Lights Chess! ♟️